

Enhanced Coinduction at Scale: Refactoring Interaction Trees from Parameterized Coinduction to the Tower

Anonymous Author(s)

Abstract

Coinductive reasoning is central to mechanized semantics, and the Rocq Proof Assistant offers several abstractions to support it. Parameterized coinduction (paco) underlies the Interaction Trees (ITrees) library, while enhanced coinduction, most recently formulated through the tower induction principle over the companion, promises greater simplicity and better compositionality of up-to techniques. We evaluate enhanced coinduction at scale by refactoring the ITrees library from paco to a tower-based enhanced coinduction library. The exercise revealed weaknesses in the existing enhanced coinduction implementation and prompted fully rewriting it. The new library eliminates the OCaml plugin in favor of a pure Gallina and Ltac formulation, reduces the codebase by 62%, extends documentation and test coverage, and fixes several latent bugs along the way. Re-integrating the rewritten library into ITrees validates the approach.

gap. “Coinductive proofs shrink by N% in lines of code, and individual lemmas that previously required multi-step paco incantations now follow from a single application of tower induction.” Data now exists for this sentence. Measured: proof text falls 8.3% in `theories/` and 11.7% across the whole development; proof *lines* fall only 1.6% and 4.1%, so the lines-of-code framing understates it by about 3× and characters is the stronger number. Coinduction-specific tactic names go 16 → 11 in `theories/`, and the Ltac implementing them falls 45.4%. Figure 1 is a candidate for the “single application of tower induction” half.

These results suggest that tower-based enhanced coinduction is not only a theoretical simplification but a practical improvement for large coinductive developments. We describe the design decisions behind the rewrite and draw lessons for coinduction library design in Rocq.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. CPP '27,

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

note. “reduces the codebase by 62%” needs a referent, and the closest measure now reads 65%. The tactic implementation, meaning `tactics.v` plus the OCaml plugin it called, goes 229 + 307 = 536 lines to 188, which is −65%. The library as a whole goes the other way, 1395 → 1485 core lines (+6%), because tests grew 84 → 328 and comments 370 → 455. Counting the deleted plugin against the old total, 1702 → 1485 is −13%. So 62% is defensible for the tactic implementation and is now conservative; it is not defensible for “the codebase”. Table 1 has all three framings.

1 Introduction

Coinductive techniques... Such techniques exist in proof assistants at two levels: foundational and abstracted. Foundational is paco/gpaco [3, 8], enhanced coinduction [4, 5]; abstracted is `itrees` [6] and `ctrees` [2].

Larger developments are often built off of abstraction, but sometimes use low-level reasoning (Vellvm [7] using paco proofs).

We rewrote `itrees` from paco to enhanced coinduction, expecting large improvements.

Some improvements, but disappointing.

So we rewrote the enhanced coinduction library to upstream a lot of desired behavior that was duplicated in `itrees`.

With this, we saw a

gap. %

improvement.

note. Pick which number carries the headline here. Candidates, all measured: tactic characters in `theories/` −9.4% against the paco baseline (and −56% against the first port); proof characters −8.4% in `theories/`, −11.8% overall; build time recovering from +25.6% to +4.1%.

The documentation of this study, and the new enhanced coinduction and `itrees` libraries, are our primary contribution.

note. Add an explicit contributions list at the end of this section.

CPP reviewers look for one. The four that the material supports: (1) the refactoring itself, (2) the negative result from the first port, (3) the rewritten library, (4) the three-checkpoint quantitative evaluation.

2 Notation, Terminology, and Background

We fix some standard notation across the paper.

Coinduction based on greatest fix points of monotone functions. We denote greatest fix points as gfp , and an arbitrary monotone function as b .

Paco is parameterized coinduction; tower is the tower induction approach of Schäfer and Smolka [5] adapted by Pous [4].

2.1 Bisimilarity

Up-to techniques are functions that satisfy this law:

gap. State the compatibility law. $f \circ b \leq b \circ f$.

This lets us write an easier coinductive proof.

gap. The easier proof. Smallest example that shows it.

2.2 Interaction Trees

An Interaction Tree (ITrees) is a coinductive data structure for representing impure and diverging computation inside a proof assistant (which is necessarily pure and terminating). The full details of ITrees are too complex to cover in full in this paper; the curious reader may inspect the original Interaction Trees paper [6].

It suffices to say that the definitions of strong and weak bisimilarity over itrees, and the proofs depending on those definitions, rely heavily on coinductive machinery. Interaction Trees was originally constructed using paco, as it enabled

gap. ...

2.3 Problems with paco

But paco has problems and/or warts:

- Many tactics, and many duplicates across arities.
- Hard to read and write compatibility proofs:

gap. example

Theory: enhanced coinduction, with its fewer tactics, all-arity story, and uniform compatibility proofs, will be a simpler theory to underly itrees and will lead to shorter proofs.

This is important because abstractions leak, as mentioned above. When they do, we want an easy-to-use theory!

3 Methodology

note. This section is a placement, not new content. It holds the measurement paragraph that sat above the abstract in the outline, which is methodology and belongs here. The text below is that paragraph verbatim.

Metric is how many paco-specific newly-needed tactics there were before, and how many of that kind for the new

library there are now. We should measure this in lines and characters of ltac for each of these as well.

We also want total lines of proof and definition, as well as characters of proof and definition.

The three checkpoints are: paco version, old port, new port.

note. What the released measurement actually does, for whatever of this you want to keep. *Checkpoints:* paco is ITrees on paco; old port is the port against the released library; new port is the port against the rewritten one. Two scopes are reported separately, *theories/* alone and *theories/ + extra/*, because they move in opposite directions on tactic surface. *Tactic counting* uses three conventions: the unit is the name a user types (**step** and **step in** are one name); a private helper folds into the name it serves; a name defined in several files counts once, with every copy counted in the line and character totals. Membership is curated by hand, not grepped, because a name-based scan was wrong three times in this work. *Lines* are counted comment-free with whitespace stripped; blank lines do not count. *Note that Ltac is specification for the classifier*, so tactic definitions are counted inside the definition columns. Several claims in Section 5 depend on separating them out.

note. Build time is measured and is not in the outline. Consider a subsection here for the protocol, since it changed the answer three times: `make TIMED=1` leaves timing artifacts in one tree and not another; blocked ordering plus machine warm-up manufactures a difference that is not real, so runs must interleave; and a single run cannot resolve a two-second effect on a ninety-second build. Results are in Table 3.

4 A Large-Scale Experiment

We began our study with a theory: that enhanced coinduction enables writing shorter proofs and simpler definitions and theorems than paco.

We put this to the test in the itrees library, and refactored it with enhanced coinduction.

ITrees was this many LOC, most of which we touched:

gap. The size figure. Measured: 96 files, 8 853 definition lines and 12 018 proof lines at the paco checkpoint across *theories/* and *extra/*. Table 2 has the full breakdown.

5 Initial Result

At a first glance, felt positive: removing lots of compatibility code, fewer tactics, arities down.

We expected great improvements to proofs, and indeed, many proofs are greatly simplified:

gap. The outline leaves this example blank. Figure 1 is a candidate, picked because the lemma statement is character-for-character identical at the two checkpoints so only the proof differs. Swap it if a better one comes to mind; the largest proof-text reductions on identical

Under PACO

```

221 Lemma eqit_inv_Tau b1 b2 t1 t2 :
222   eqit RR b1 b2 (Tau t1) (Tau t2) → eqit RR b1 b2 t1 t2.
223 Proof with eauto with itree.
224   intros. punfold H. red in H. simpl in *.
225   remember (TauF t1) as tt1. remember (TauF t2) as tt2.
226   hinduction H before b2; intros; try discriminate.
227   - inv Heqtt1. inv Heqtt2. pclearbot. eauto.
228   - inv Heqtt1. inv H.
229     + pclearbot. punfold REL. pstep. red. simpobs...
230     + pstep. red. simpobs. econstructor; eauto.
231     + pstep_reverse. apply IHeqitF; eauto.
232     + eauto with itree.
233   - inv Heqtt2. inv H.
234     + pclearbot. punfold REL. pstep. red. simpobs...
235     + eauto with itree.
236     + pstep. red. simpobs. econstructor; auto.
237     + pstep_reverse. apply IHeqitF; eauto.
238 Qed.

```

Under enhanced coinduction

```

240 Lemma eqit_inv_Tau t1 t2 :
241   eqit RR b1 b2 (Tau t1) (Tau t2) → eqit RR b1 b2 t1 t2.
242 Proof.
243   intros.
244   step in H; step.
245   now apply eqitF_inv_Tau.
246 Qed.

```

Figure 1. The same lemma at the PACO and new port checkpoints. The statement is identical; only the proof differs. The PACO proof interleaves coinductive plumbing with an induction on the derivation, and six of its tactic calls (punfold, pclearbot, pstep, pstep_reverse) exist only to move between PACO’s guarded and unguarded forms. The port separates the two. Taking one step in the hypothesis and one in the goal is **step in** H; **step**, and the induction becomes eqitF_inv_Tau, an ordinary 19-line lemma about the one-step functor that contains no coinduction at all and is reused elsewhere. The total is not much smaller. What changes is that the coinductive reasoning is three lines and is separable from everything else.

statements are proper_eqitree_throw_prefix (921 → 87 chars, but the statement changed so it is not like-for-like), eqit_inv_Tau (604 → 57, used here), and rutt_Proper_R2 (659 → 121).

But this was offset by having to write lots of workarounds to missing or insufficient definitions of tactics in the enhanced coinduction library.

Particularly **step** and a bug in coinduction led to these pain points:

gap. The pain points.

Disappointingly, we saw little reduction in overall number of lines of code. In fact, in spite of reducing 800 lines of proof, we observed a net gain in the total LOC (as calculated by rocq wc):

note. Total size never rose, at either port. Counting every non-blank comment-free line once, the whole development goes 20 871 → 20 455 → 20 156, which is −2.0% at the first port and −3.4% after the rewrite. Figure 2 plots it. The rocq wc figure the outline relied on is not sound on these files in either direction; see paper-data/rocqwc-unsound.md. What did go the wrong way at the first port is the tactic surface, which *nearly tripled*, 4 586 → 12 754 characters, +178%. That is the sentence this paragraph wants.

We can see in Figure 2 that nearly all of this stems from definitions. Why?

note. This part holds, and the mechanism is now precise. Across theories/ + extra/ the definition column rose 8 853 → 8 934 during the port, the only measure that rose at all. Separating tactic code out of that column: ordinary definitions *fell* 8 726 → 8 574, and tactic lines rose 127 → 360. The 81-line rise is −152 ordinary plus +233 tactic. Every bit of the definitional growth is tactic workaround code, which is exactly the claim below.

We identified two main reasons:

1. Some simple bugs in the coinduction library required large workarounds to fix.
2. The **step** tactic for exposing the *b* was not expressive enough, requiring large redefinitions.

Furthermore, the enhanced coinduction library contained a few technical warts that made development slow. In particular, some of the error messages were quite mysterious:

```

Goal gfp b 5 6 → gfp b 7 8.
      coinduction R H.

```

This is odd, as

gap. ...

It turns out that

gap. ...

note. For the two reasons above, the material that exists: **step** could not see through a definition standing for a fixed point, and failed at the arity up-to bind needs, so ITrees reimplemented stepping in two places. The plugin rejected any goal with the same gfp in a hypothesis and the conclusion, which is routine in ITrees. Both are pinned as compiling Rocq goals in paper-data/probes.v, claims 1–2 and 5–7. A second mysterious-message example, if one is wanted: a candidate named after a binder already in the goal failed with a message about a missing chain.

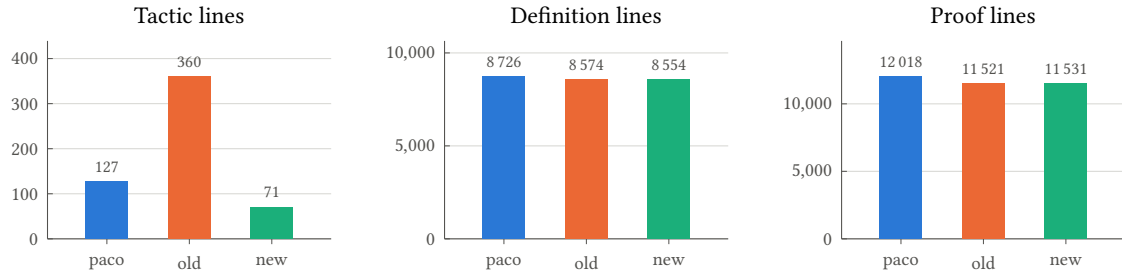


Figure 2. The whole development at the three checkpoints, counting the library proper and its four downstream users together. Each panel carries its own scale, because the three measures differ by two orders of magnitude, and every panel is zero-based. Definition lines exclude tactic definitions, which the first panel counts. Definitions and proofs barely move across the three checkpoints. Tactic code, the measure enhanced coinduction was expected to reduce, almost triples at the first port and returns below the PACO baseline only after the library was rewritten.

6 Improvement: Rewriting the Coinduction Library

Goal was to fix bugs. But OCaml plugin got in the way.

6.1 OCaml plugin: why it exists

Exists to

gap. ...

note. The reason, from the library's own comments: applying tower induction needs the goal presented as a predicate applied to the candidate, and guessing that predicate is the hard part. The released library reified the goal into a small syntax (hole, abstraction, conjunction) that is inf-closed by construction, which discharges the first obligation for free. The cost is getting back a user-readable goal: interpreting the reified syntax with Rocq's own reduction performs unwanted simplifications *and discards the user's bound-variable names*. The plugin reconstructed the goal type in OCaml by walking the original goal syntactically, purely to recover those names. That is all 307 lines were for.

6.2 Removing it

gap. ...

note. What replaced it: pattern already abstracts a goal over a subterm and preserves names because it never leaves the goal's own syntax. Tower induction becomes three lines, with the inf-closedness obligation going to proof search instead of being free by construction:

```
Ltac tower_induction_with c :=
  pattern (elem c);
  apply tower;
  [icauto | try clear_old_chain].
```

6.3 Fixing bugs became much easier

With bug fix and other error fixes, the improvement is...

gap. ...

note. The fixed bugs, for reference: gfp in a premise; accumulation failing when a premise mentioned the candidate under a function ($f \ (elem \ c)$); a candidate name clashing with a binder of the goal; **step** not seeing through a definition, and taking more than one step. Each is a compiling goal in paper-data/probes.v.

6.4 What else did I do?

Automatic monotonicity. In practice, 18 out of 19 of the stated monotonicity proofs in the Interaction Trees library can be automatically discharged using **monauto**, with the last being a complex proof involving multiple nontrivial steps. **monauto** also discharges every one of the monotonicity sub-proofs for ptower in the Interaction Trees library, which number

gap. _

note. The counts have moved; current tree says 17 of 18. There are 18 statements of the form $\text{Proper } (leq \implies leq) \ f$ in ITrees, and 17 are **Proof**. **monauto**. **Qed**. verbatim. The exception is `iter_mono` in `extra/Dijkstra/PureITreeDijkstra.v`, whose functor is packaged in a dependent record, so the proof must destruct it and apply a monotonicity field stored inside before automation can start. For the blank: **tower induction** is used at 22 sites, and `apply_ptower` sends its two side conditions to **monauto** and **icauto**. ITrees contains *zero* calls to **icauto**, which is the sharper way to say it: the obligations never reach the user.

Automatic inf-closedness.

gap. ...

note. **icauto** is **auto** over a hint database. Most cases are structural. The one that is not: for a relation class such as **Reflexive** or a **Proper** instance, the inf-closedness is visible only after unfolding, and the

Table 1. The coinduction library before and after the rewrite, in comment-free non-blank lines. The tactic implementation is the tactic file together with the OCaml plugin it called. The plugin is gone, so the rewritten library has no OCaml in it at all.

Component	Released	Rewritten	Change
<i>Tactic implementation</i>			
tactics.v	229	188	−18%
OCaml plugin	307	0	−100%
total	536	188	−65%
<i>Rest of the library</i>			
lattice theory	384	499	+30%
tower	134	150	+12%
companion	523	499	−5%
relations	124	148	+19%
Core total (excluding tests)	1 395	1 485	+6%
Comments	370	455	+23%
Tests	84	328	+290%

unfolding sits inside the predicate rather than at its head, so **Hint** Unfold cannot reach it. A **Hint** Extern does that one delta step. Claim 8 of probes.v pins this at ITrees' arity.

note. Two rewrites in the git log that the outline does not mention, and which belong in this section next to the two above: **accumulate** and **symmetric** were both reimplemented without the plugin. Accumulation now reverts every hypothesis mentioning the candidate, folds them into one conjunction, applies ptower, and restores them under their original names, which is also what let it detect and report the one conclusion shape it cannot support.

6.5 What the library cost and gained

note. This subsection is not in the outline; the table needed somewhere to live. Delete the heading and fold Table 1 wherever you prefer.

7 Results

note. This section is a container, not outline content. The outline reports results inline in Section 5 and Section 6. The measured data has to appear somewhere, so it is parked here; fold it back into those sections if you would rather not have a separate results section. Nothing below is prose, only the tables and what they say.

note. Tactic surface. The rewrite finishes well below the paco baseline at both scopes: in theories/, 16 → 11 names (−31%), 118 → 66 lines (−44%), 4 181 → 2 283 characters (−45.4%); across the whole development, 4 586 → 2 545 characters (−44.5%). The first port went the wrong way on every one of them. Ten tactic families disappear from theories/ outright between the two ports; three of them (inf_closed_auto, monauto, tower induction) went because

the library's own versions became good enough to reach the files that needed them, and the to_mon family collapsed into a single shared driver.

note. The downstream developments now cost less than they did under paco. extra/ carried 405 characters of coinduction-driving Ltac at the paco checkpoint, 4 217 after the first port, and 262 now. The reason is that the per-functor to_mon blocks, which were the whole of the regression, are gone: each development states one **reflexivity** lemma and registers it, and a single driver in Basics/Utils.v does the folding. This is the clearest single piece of evidence that the first port's cost was implementation and not theory.

note. Characters move about three times as far as lines. Proof lines fall 4.1% across the whole development while proof characters fall 11.8%; in theories/, 1.6% against 8.4%. Enhanced coinduction did not mainly delete proof lines, it made the surviving lines shorter. This is the argument for reporting both, and it is a finding the paper would have missed on a line count alone.

note. The two halves moved in opposite directions. extra/ gained more proof reduction than theories/ did, and also paid the most tactic surface to adopt the library. The downstream users had the most to gain and the most to pay.

note. Build time (Table 3). The first port cost +25.6% against paco; the rewrite recovers it to +4.1%. Diagnosed causes of the first port's regression, none inherent to enhanced coinduction: the port dropped the explicit eauto depth annotations the paco code carried, leaving the default depth whose last two levels go to the standard library's boolean machinery; inv gained a try easy and is called at 1118 sites; and 33 sites changed hinduction ... before r to ... before E, reverting the whole context instead of the tail.

8 Related and Future Work

Paco, gpaco [3, 8].

Enhanced coinduction, the tower [4, 5].

note. Also worth citing here: the diacritical companion [1], which is the passive/active distinction the future-work item below builds on; ctrees [2] as the contrast case, a development built on enhanced coinduction from the start rather than migrated; and Vellvm [7] for the leaking-abstraction argument in Section 1. All are in refs.bib already.

8.1 Future work

Theoretical strengthening of the coinduction library: active up-to techniques.

note. The extra/IForest.v deduplication that an earlier draft listed here as future work is done, and its result is reported in Section 7.

Table 2. The three checkpoints on every measure. *Tactics* counts distinct coinduction-driving tactic names; *lines* and *chars* give the size of their definitions, every copy counted. Definition figures include tactic definitions, since `Ltac` is specification for the classifier. *main* is the library proper; *main+extra* adds the four downstream developments.

Checkpoint	Scope	Coinduction tactics			Definitions		Proofs	
		names	lines	chars	lines	chars	lines	chars
paco	main	16	118	4 181	5 870	213 716	6 231	176 031
paco	main+extra	20	127	4 586	8 853	354 025	12 018	399 700
old port	main	21	240	8 537	5 820	213 284	6 119	160 788
old port	main+extra	23	360	12 754	8 934	355 226	11 521	351 873
new port	main	11	66	2 283	5 613	206 132	6 133	161 372
new port	main+extra	12	71	2 545	8 625	344 902	11 531	352 792

Table 3. Wall-clock time for a full serial build of Interaction Trees at each checkpoint, in seconds. Builds are interleaved rather than blocked, and the coinduction library is prebuilt, so the timed region is the Interaction Trees development itself. Mean over 4 rounds, and over the 3 rounds after the cold first one.

Checkpoint	All rounds		Warm rounds	
	mean	vs. PACO	mean	vs. PACO
paco	85.4	—	84.7	—
old port	106.9	+25.2%	106.4	+25.6%
new port	88.9	+4.1%	88.2	+4.1%

9 Conclusion

We rewrote itrees and the coinduction library to fix bugs, go to a Rocq-native implementation that is easier to debug for intermediate users, and get better test coverage.

Improved abstractions.

note. Test coverage, for the claim above: `tests.v` goes 84 → 328 comment-free lines, and comments across the library 370 → 455. Both are in Table 1.

Data-Availability Statement

note. Not in the outline; added because CPP has an artifact track and reviewers look for this. Draft text below, to keep or cut.

The rewritten coinduction library, both ports of Interaction Trees, the measurement script, the raw data behind every table in this paper, and the file of behavioural claims stated as compiling Rocq goals are available as supplementary material.

gap. Archival DOI for the camera-ready.

References

- [1] Dariusz Biernacki, Serguei Lenglet, and Piotr Polesiuk. 2019. Diacritical Companions. *Electronic Notes in Theoretical Computer Science* 347 (2019), 25–43. doi:10.1016/j.entcs.2019.09.002 Proceedings of MFPS XXXV.

- [2] Nicolas Chappé, Paul He, Ludovic Henrio, Yannick Zakowski, and Steve Zdancewic. 2023. Choice Trees: Representing Nondeterministic, Recursive, and Impure Programs in Coq. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 61 (2023). doi:10.1145/3571254
- [3] Chung-Kil Hur, Georg Neis, Derek Dreyer, and Viktor Vafeiadis. 2013. The power of parameterization in coinductive proof. In *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 193–206. doi:10.1145/2429069.2429093
- [4] Damien Pous. 2016. Coinduction All the Way Up. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. ACM, 307–316. doi:10.1145/2933575.2934564
- [5] Steven Schäfer and Gert Smolka. 2017. Tower Induction and Up-to Techniques for CCS with Fixed Points. In *Relational and Algebraic Methods in Computer Science (RAMiCS) (Lecture Notes in Computer Science, Vol. 10226)*. Springer, 274–289. doi:10.1007/978-3-319-57418-9_17
- [6] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. 2020. Interaction Trees: Representing Recursive and Impure Programs in Coq. *Proceedings of the ACM on Programming Languages* 4, POPL, Article 51 (2020). doi:10.1145/3371119
- [7] Yannick Zakowski, Calvin Beck, Irene Yoon, Ilia Zaichuk, Vadim Zaliva, and Steve Zdancewic. 2021. Modular, compositional, and executable formal semantics for LLVM IR. *Proceedings of the ACM on Programming Languages* 5, ICFP, Article 67 (2021). doi:10.1145/3473572
- [8] Yannick Zakowski, Paul He, Chung-Kil Hur, and Steve Zdancewic. 2020. An Equational Theory for Weak Bisimulation via Generalized Parameterized Coinduction. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. ACM, 71–84. doi:10.1145/3372885.3373813